



Comparing protein sequence alignment score networks

Davin (Haoping) Qiu

Supervisor: Karel Devriendt

Mathematical Institute
University of Oxford

September 2026

Abstract

Sequence-similarity networks (SSNs) encode protein sequences as vertices and selected pairwise similarities as edges. Their topology depends on the protein sample, the score method, and the rule that turns scores into edges. We systematically compare these methodological choices and evaluate how well the resulting graph structure reflects Pfam protein families and clans. We implemented a reproducible pipeline on 960 Pfam domains from 24 families and 8 clans. Using a factorial design, we sampled 900 balanced collections and built 21,600 weighted graphs from exact local alignment or BLAST scores under three edge rule families, then detected communities with the Louvain method. We measured recovery of withheld Pfam labels by adjusted mutual information (AMI). Overall, top- k rules, which connect each domain to its k highest-scoring neighbours, recovered Pfam structure most consistently under both score methods; top-10 reached median family AMI 0.952 with exact alignment and 0.928 with BLAST. At matched 2% edge density, median absolute family normalised mutual information (NMI) difference between methods was 0.0034, whereas equal score percentiles produced unequal densities. Score permutation reduced family AMI at reference setting from 0.936 to 0.009. Results quantify sampling and edge selection effects in protein network analysis.

1 Introduction

Proteins perform most molecular functions in living cells, including catalysis, signalling, and transport. Proteins that share ancestry often share structure and function, so relating proteins through their sequences is a basic task in molecular biology. We may consider a protein as a finite ordered sequence of amino-acid residues and intuitively would expect proteins with similar sequences to be similar in some biological sense. Protein alignment is then the family of techniques used to estimate “similarity” from amino acid sequences alone [Durbin et al., 1998]. Protein sequences are typically long, so we instead focus on protein domains: shorter, contiguous subsequences of a protein.

For any given pair of protein domain sequences of comparable length, pairwise alignment inserts gaps into two such sequences so that residues judged to correspond occupy shared positions; a gap marks residues present in one sequence and absent in the other, an indel whose direction pairwise alignment does not determine. A substitution matrix scores each aligned residue pair and gap costs penalise indels, so total score rewards correspondence and penalises divergence. Global alignment scores two sequences over their full length; local alignment selects one region of each sequence maximising total score, and so tolerates unrelated flanking segments. Comparison at domain level uses local alignment. Several algorithms produce such a score, differing in computational cost and in which pairs they report; Section 2 discusses them further.

One application of pairwise alignment scores is the construction of sequence-similarity networks (SSNs), in which vertices represent sequences and selected pairs form weighted edges. SSNs support exploration and clustering of protein families [Atkinson et al., 2009, Gerlt et al., 2015, Enright et al., 2002]. Sequences alone do not determine an SSN. Three choices intervene: the collection of proteins considered, the alignment algorithm to score pairs, and the graph rule that promotes scores to edges. Threshold choice has no accepted universal value, and reported topology shifts with it [Apeltsin et al., 2011, Hornung and Terrapon, 2023].

In this article, we conduct a systematic comparison of methodological choices in SSN construction. For graphs built from different protein collections, alignment algorithms, and graph construction rules, we evaluate how well the resulting graph structure reflects known biological protein families.

We vary these three choices systematically on a recorded panel of 960 seed domains from 24 families and 8 clans of the Pfam database [Mistry et al., 2021]. Using a factorial design, we sample 900 balanced collections, and across two scoring methods and 12 edge settings we obtain 21,600 weighted graphs. We then detect communities in each graph without using labels and quantify their agreement with withheld Pfam family and clan labels.

2 Alignment algorithms and scoring

In this section, we describe the two methods we use to score pairs of sequences: exact local alignment by the Smith–Waterman algorithm and heuristic search by BLAST. We then compare their scoring systems and briefly describe a learned alternative that we did not use at scale.

2.1 Exact local alignment

Let $x = x_1 \dots x_p$ and $y = y_1 \dots y_q$ be amino-acid sequences, where each x_i and y_j is one of the 20 standard amino acids. A substitution matrix $a(\cdot, \cdot)$ assigns score $a(x_i, y_j)$ to an aligned residue pair. BLOSUM62 is a 20×20 substitution matrix estimated from conserved blocks of aligned proteins, its entries scaled log-odds of observed against chance residue pairing [Henikoff and Henikoff, 1992].

We use affine gap cost [Gotoh, 1982], where opening a gap costs $g_o > 0$ and each further gap residue costs g_e , with $0 < g_e < g_o$, so a gap of length ℓ costs $g_o + (\ell - 1)g_e$. One long gap therefore costs less than several short gaps of equal total length, since a single multi-residue indel occurs more readily than several isolated ones. We take $g_o = 11$ and $g_e = 1$, conventional values for BLOSUM62.

The Smith–Waterman algorithm finds an exact optimal local alignment by dynamic programming [Smith and Waterman, 1981]. Gotoh’s formulation computes this optimum under affine gap costs in $O(pq)$ time by carrying separate states for gapped and ungapped extensions,

$$\begin{aligned} E_{ij} &= \max\{H_{i-1,j} - g_o, E_{i-1,j} - g_e\}, \\ F_{ij} &= \max\{H_{i,j-1} - g_o, F_{i,j-1} - g_e\}, \\ H_{ij} &= \max\{0, H_{i-1,j-1} + a(x_i, y_j), E_{ij}, F_{ij}\}. \end{aligned}$$

Here H_{ij} is best score of a local alignment ending with x_i aligned to y_j ; E_{ij} is best score ending with x_i aligned to a gap in y ; F_{ij} is best score ending with a gap in x aligned to y_j . Boundary values are $H_{i0} = H_{0j} = 0$, with gap states unavailable at a boundary set to $-\infty$. Zero inside H_{ij} lets an alignment restart, which yields a local rather than global optimum. Under this recurrence a gap of length ℓ costs $g_o + (\ell - 1)g_e$, and local score is

$$s(x, y) = \max_{i,j} H_{ij}.$$

In this article, we compute exact local alignment scores with the Smith–Waterman implementation in Biopython’s PairwiseAligner [Cock et al., 2009], run in local mode with BLOSUM62 and affine gap costs $g_o = 11$ and $g_e = 1$.

2.2 Heuristic search

BLASTP, the protein–protein search program of BLAST, avoids exhaustive dynamic programming. It indexes short words, locates high-scoring word matches between query and subject, and extends only those seeds into local alignments [Altschul et al., 1990, Camacho et al., 2009]. Cost per pair falls well below $O(pq)$. A weakly similar pair may yield no seed and therefore no reported alignment, so absence from BLAST output records failure to reach a reporting threshold rather than a score of zero.

Raw score S carries units of its scoring system. Bit score rescales it to a scale independent of scoring system,

$$S' = \frac{\lambda S - \log K}{\log 2},$$

where λ and K are Karlin–Altschul parameters fixed by substitution matrix, gap costs, and background residue frequencies [Karlin and Altschul, 1990]. In this article, BLAST graphs use bit scores S' as edge weights.

We ran BLAST using BLOSUM62 and nominal gap costs $g_o = 11$ and $g_e = 1$, matching exact local alignment of Section 2.1, so both methods score aligned residue pairs from one substitution matrix. BLAST additionally applies composition-based statistics in mode 2, which rescales substitution scores separately for each query–subject pair according to that pair’s residue composition. Rescaling suppresses high scores arising from compositional bias rather than from shared ancestry, for instance between two sequences that are both rich in a single residue.

BLAST reports an alignment only when its expect value falls below a threshold. For an alignment with raw score S and bit score S' , the E-value of the pair is defined as

$$E = Km'n' e^{-\lambda S} = m'n' 2^{-S'},$$

where m' and n' are effective query and database lengths after correction for alignments truncated at sequence ends, and λ , K are as above. Both forms agree because $2^{-S'} = Ke^{-\lambda S}$, the bit score being defined precisely so that expect value takes this form. Hence E counts alignments expected to reach score S by chance alone in a search of that size, and small E marks a score unlikely to arise at random. Since E scales with $m'n'$, one value of E denotes different degrees of similarity in databases of different size.

We used $E = 10$, the standard BLAST default. That value is permissive by design, far above thresholds near 10^{-3} conventionally used to claim homology, and it exists to surface marginal hits rather than to establish significance. Permissiveness is apt here since edge selection happens downstream under rules of Section 3.4, leaving E to determine only which pairs carry a score at all.

BLAST searches are directional and may return several high-scoring segment pairs (HSPs) for one query–subject comparison. HSP of highest bit score represented each ordered pair, and highest score across reported directions represented each unordered pair. In our experiments, BLAST reported 19,376 of 460,320 master pairs, or 4.21%.

2.3 Comparability of the two scoring systems

Both methods used BLOSUM62 and nominal gap parameters $g_o = 11$ and $g_e = 1$. Two differences remain. BLAST charges an extension cost for every gap residue, so a gap of length ℓ costs $g_o + \ell g_e$, exceeding cost under exact alignment by g_e at every gap. Composition-based statistics further adjust BLAST substitution scores per sequence pair, while exact local alignment applies one fixed matrix throughout.

Each method therefore ran at its own standard default. Method comparisons in this report control vertex set and edge count (Section 3.4) rather than assert identical scoring systems.

2.4 Learned alignment

DEDAL is a pretrained neural model that learns sequence representations and alignment parameters from protein data, returning a Smith–Waterman-style score [Linares-López et al., 2023]. We verified this pretrained model on pilot data, but its measured runtime exceeded our compute budget for all-versus-all scoring of the master panel (Section 3.2), so scaled results use exact local alignment and BLAST only.

3 Methods

In this section, we describe how we turn pair scores into graphs and evaluate them. We first define networks built from pair scores, then describe how we select a panel of Pfam domains and sample collections from it, how edge rules select edges, and how we detect communities and compare them with Pfam labels.

3.1 Networks from pair scores

A collection C is a set of protein domain sequences drawn from the master panel (Section 3.2). Each graph built from C has one vertex per sequence, so we identify its vertex set V with C . Let $n = |V|$, $\mathcal{P}(V) = \{\{i, j\} : i, j \in V, i \neq j\}$, and $P = |\mathcal{P}(V)| = \binom{n}{2}$. Method m reports score s_e^m for each pair in available set $A_m \subseteq \mathcal{P}(V)$; pairs outside A_m stay missing and never receive an imputed value. Edge setting θ selects $E_{m,\theta} \subseteq A_m$, giving weighted graph

$$G(C, m, \theta) = (V, E_{m,\theta}, w_m), \quad w_m(e) = s_e^m,$$

with realised density $\delta = |E_{m,\theta}|/P$, fraction of possible pairs that became edges. Term *realised* marks δ as an outcome rather than an input, since top- k and percentile rules fix no edge count in advance and target density can fall short of its request. Every input sequence remains a vertex; sequences with no selected pair become isolated vertices.

Each vertex carries a stable identifier formed from its UniProt accession with domain start and end coordinates, for example A0A0H3GJZ1_27_153. Lexicographic order of identifiers resolves ties wherever a selection would otherwise be ambiguous, making every procedure below deterministic.

The path from collection to reported quantity is

$$C \longrightarrow \{s_e^m : e \in A_m\} \longrightarrow G(C, m, \theta) \longrightarrow U(C, m, \theta) \longrightarrow \text{AMI}(U, V^*),$$

where U is a community partition detected without labels and V^* is withheld Pfam partition. The remaining subsections define each arrow. The master panel and sampling design fix C , edge rules fix θ , and community detection with agreement measures fixes U and final comparison.

3.2 Master panel

To sample subsets of protein sequences on which to build graphs, we first fix a master panel: a recorded set of Pfam seed domains from which every collection is drawn. The Pfam database defines families of homologous domains using curated seed alignments and profile hidden Markov models; clans group families with evidence of common ancestry [Mistry et al., 2021].

We select the panel by a rejection sampling procedure over Pfam clans. The procedure first reads the official family–clan catalogue, discards unassigned families and clans holding fewer than three catalogue families, then shuffles the sorted clan list. Within each clan in that order it shuffles the sorted family list and inspects families in turn, accepting those of InterPro type *domain* or *family* carrying at least 40 reported seed domains. A clan is retained once three families are accepted, and skipped when fewer prove eligible. Selection stops at 8 retained clans. One random stream drives every shuffle, so a single seed reproduces the panel. A trace table records every family inspected together with its acceptance or rejection. Pseudocode of the full procedure may be found in Appendix A.

Each selected seed alignment is filtered to unique sequences over 20 standard amino acids with consistent coordinates. We then use deterministic farthest-point sampling to select 40 representatives per family. First sequence has length nearest family median. Given selected set S and remaining set R , next sequence is

$$x^* = \arg \max_{x \in R} \min_{y \in S} d(x, y), \quad d(x, y) = 1 - \text{identity}(x, y),$$

where identity counts matching residues over jointly occupied seed alignment positions. Identifier order resolves ties. This maximises within-family sequence diversity, so recovery results are conservative relative to a sample of each family weighted by frequency. Alignment gaps are removed before scoring. Panel contains 960 domains, 24 families, 8 clans, and $\binom{960}{2} = 460,320$ unordered pairs.

3.3 Repeated collections

Each collection is a balanced hierarchical draw. For condition (c, f, r) , sample c clans uniformly without replacement, then f families within each selected clan, then r domains within each selected family. Collection size is $n = cfr$, with equal family sizes by construction. Factor levels are

$$c \in \{2, 4, 8\}, \quad f \in \{2, 3\}, \quad r \in \{10, 20, 40\}.$$

Cartesian product gives $3 \cdot 2 \cdot 3 = 18$ conditions, and 50 seeded replicates per condition give $18 \cdot 50 = 900$ collection declarations. A stable hash of base seed, condition, and replicate index generates each replicate seed, so extending the design leaves earlier collections unchanged. Pfam labels determine sampling strata and are then withheld; they enter no score, edge rule, or community detection.

Conditions requesting most of the panel can repeat a complete node set across replicates despite distinct seeds. A uniqueness audit hashes sorted node identifiers within each condition. 4 conditions contain duplicates; their 200 declarations remain in audit tables and figures but leave distributional

summaries. Primary distributions therefore use 700 collections from 14 conditions with 50 distinct node sets each. All 4 excluded conditions request $r = 40$, so exclusion falls entirely on largest domain count and composition effects reported below are conservative. Two score methods and 12 edge settings generate 21,600 graphs before this filter.

3.4 Edge rules

Recall vertex set V of collection C , possible pairs $\mathcal{P}(V)$ with $P = |\mathcal{P}(V)|$, and available set $A_m \subseteq \mathcal{P}(V)$ of pairs that method m scores. Edge setting θ is a rule family together with its parameter, taking one of forms (top, k), (pct, p), or (dens, d) defined below.

For $i \in V$, the available neighbours are $A_m(i) = \{j \in V : \{i, j\} \in A_m\}$. Let $N_k^m(i)$ be the first $\min\{k, |A_m(i)|\}$ elements of $A_m(i)$ under descending score with ties broken by ascending identifier. For $e \in A_m$, let $r_e^m \in (0, 1]$ be the ascending rank of s_e^m , with ties averaged and divided by $|A_m|$, and let q_e^m be the descending rank, with $q_e^m = 1$ at highest score. Let $b_m(d) = \min\{|A_m|, \lceil dP \rceil\}$. Three rule families are

$$\begin{aligned} E_{m,k}^{\text{top}} &= \{e = \{i, j\} \in A_m : j \in N_k^m(i) \text{ or } i \in N_k^m(j)\}, \\ E_{m,p}^{\text{pct}} &= \{e \in A_m : r_e^m \geq p\}, \\ E_{m,d}^{\text{dens}} &= \{e \in A_m : q_e^m \leq b_m(d)\}. \end{aligned}$$

$E_{m,k}^{\text{top}}$ (top, k) lets each vertex nominate up to k strongest available neighbours and retains undirected union, so retained edge count grows with n rather than with $\binom{n}{2}$ [Camoglu et al., 2006]. $E_{m,p}^{\text{pct}}$ (pct, p) retains the upper tail of scores a method reported, so it acts on A_m ; it coincides with target density $1 - p$ exactly when $A_m = \mathcal{P}(V)$, and otherwise yields a sparser graph in proportion to missing coverage. $E_{m,d}^{\text{dens}}$ (dens, d) requests a common global edge budget $\lceil dP \rceil$ on identical vertex sets, with shortfall $\max\{0, \lceil dP \rceil - |A_m|\}$ recording a budget a method could not supply. The parameters used in our experiments are

$$k \in \{1, 2, 5, 10\}, \quad p \in \{0.90, 0.95, 0.98, 0.99\}, \quad d \in \{0.005, 0.01, 0.02, 0.05\},$$

giving 12 settings per method.

3.5 Communities and agreement

To find groups of vertices that are densely connected to each other, we apply community detection to each graph. Several methods exist; we use the Louvain method. We identified communities in the graphs using weighted Louvain optimisation [Blondel et al., 2008], as implemented in NetworkX [Hagberg et al., 2008]. This heuristic starts with each vertex in its own community and alternates two phases. First, it moves single vertices into a neighbouring community whenever the move increases weighted modularity; second, it merges each community into a single vertex and repeats on the reduced graph, stopping when no move increases modularity. Weighted modularity is

$$Q = \frac{1}{2W} \sum_{i,j} \left(w_{ij} - \gamma \frac{k_i k_j}{2W} \right) \mathbf{1}\{u_i = u_j\}.$$

For vertices i and j , w_{ij} is their edge weight when $\{i, j\} \in E$, and zero otherwise. Weighted degree is $k_i = \sum_j w_{ij}$, and total edge weight is $W = \frac{1}{2} \sum_{i,j} w_{ij}$. The parameter $\gamma > 0$ controls community resolution; all experiments use $\gamma = 1$. Variable u_i denotes the community assigned to vertex i , hence indicator $\mathbf{1}\{u_i = u_j\}$ equals one when i and j share a community.

To compare detected community structure with ground-truth Pfam family and clan labels, we use two measures: normalised mutual information (NMI) and adjusted mutual information (AMI). Choose one of n vertices uniformly. Let U be its detected community and V^* its withheld Pfam label. Let n_{uv} count vertices assigned to community u with label v , $n_u = \sum_v n_{uv}$, and $n_v = \sum_u n_{uv}$. Joint and marginal proportions are

$$p_{uv} = \frac{n_{uv}}{n}, \quad p_u = \frac{n_u}{n} = \sum_v p_{uv}, \quad p_v = \frac{n_v}{n} = \sum_u p_{uv}.$$

Define

$$I(U; V^*) = \sum_{u,v} p_{uv} \log \frac{p_{uv}}{p_u p_v}, \quad H(U) = - \sum_u p_u \log p_u, \quad H(V^*) = - \sum_v p_v \log p_v,$$

with $0 \log 0 = 0$; $\log$ denotes natural logarithm. NMI and AMI are defined as

$$\text{NMI}(U, V^*) = \frac{I(U; V^*)}{[H(U) + H(V^*)]/2},$$

$$\text{AMI}(U, V^*) = \frac{I(U; V^*) - \mathbb{E}[I(U; V^*)]}{[H(U) + H(V^*)]/2 - \mathbb{E}[I(U; V^*)]}.$$

Expectation averages mutual information over random contingency tables with observed row and column margins fixed. NMI lies in $[0, 1]$ and rises with partition count even under independence, since a finer partition raises $I(U; V^*)$ by chance alone. AMI subtracts that expectation, takes value 0 in expectation under independence, and so permits comparison across settings whose community counts differ [Vinh et al., 2010]. We computed both NMI and AMI using scikit-learn [Pedregosa et al., 2011]. We stress again that family and clan labels entered evaluation only after community detection.

3.6 Reproducibility and figure provenance

Stored graph tables carry one row per graph and reference label level, and all reported analyses read those tables. Script `scripts/build_report_assets.py` generates every figure and numerical macro from saved TSV and JSON artifacts. Canonical PNGs are written under `outputs/figures/pfam_large_panel_report/`; identical copies are placed in `report/figures/`. File `report/figure_provenance.tsv` maps each image to its generating script, exact input tables, both paths, and SHA-256 checksum. Report prose imports generated macros from `report/generated_results.tex`, so no numerical result is transcribed by hand.

4 Results

Our analysis centres on which edge rule yields communities closest to withheld Pfam structure. Agreement with Pfam labels serves as a proxy, however limited, for how well graph structure reflects biological information. We compared each Louvain partition separately with family and clan assignments.

We evaluate each tested setting by median AMI across 700 distinct primary collections; NMI shows the effect of chance correction. We report medians rather than means because both measures are bounded and their replicate distributions are asymmetric: several settings concentrate against 0 or 1, and some are multimodal (Figs. 5 and 6). The median is less sensitive to this skew and to outlying replicates. Figures report interquartile ranges across collections as error bars. Detailed distribution plots use reference condition $(c, f, r) = (4, 2, 20)$: 50 collections, each containing 160 domains from 8 families.

4.1 Graph-rule recovery of Pfam structure

Figure 1 gives median AMI against median realised density. Family recovery increased from top-1 to top-10 under both score methods. Top-10 gave highest tested family AMI: 0.952 for Biopython and 0.928 for BLAST. Corresponding clan AMI was 0.772 and 0.734. Biopython percentile 0.90 gave clan AMI 0.778, exceeding Biopython top-10 by 0.006; its BLAST counterpart gave 0.166. Top-10 was therefore first or within 0.006 of first for both label levels and both score methods.

Top-5 supplied a lower-density operating point. Median family AMI was 0.938 for Biopython and 0.917 for BLAST at densities 4.18% and 3.87%. Median clan AMI was 0.731 and 0.705. Subsequent examples use top-5 to retain fewer edges; the smaller of the two median family AMI values is 0.917.

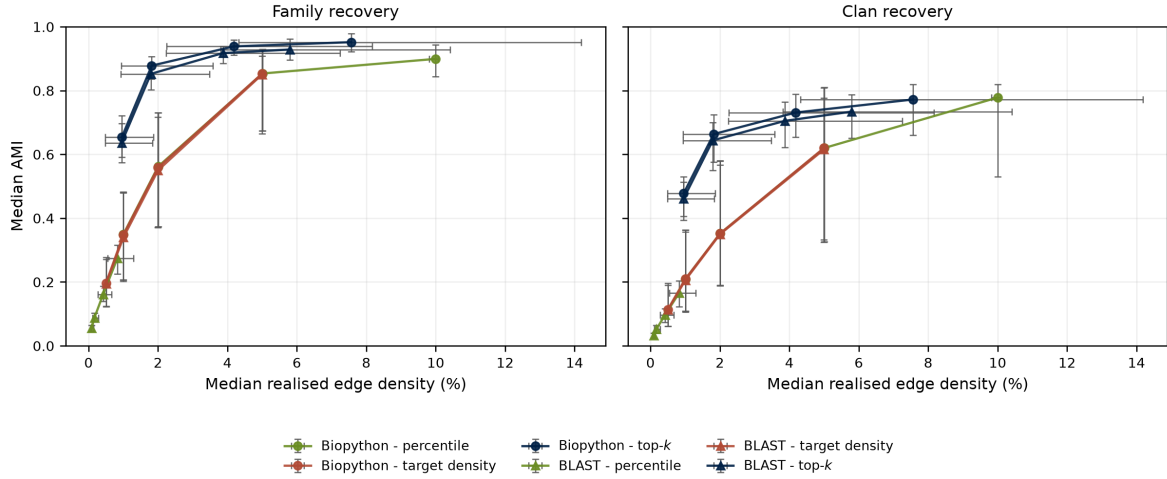


Figure 1: Median AMI against median realised density across 700 primary collections. Panels show family and clan recovery. Lines connect parameter values within each method and rule family. Error bars give interquartile ranges across collections in both coordinates. Marker denotes score method: circle for Biopython, triangle for BLAST.

Figure 2 shows one reference-condition top-5 example. Each panel contains 160 vertices from 8 families. Biopython retained 564 edges in 8 connected components; BLAST retained 558 edges in 7 components. Both family AMI values round to 0.958, although component structures differ.

Each vertex represents one domain; colour denotes withheld Pfam family. Each island is a connected component. Components are packed into a grid, then vertices within each component receive a spring layout. Shared coordinates permit vertex-wise comparison between panels; distance between islands carries no analytical meaning.

4.2 Effect of sparsity on structure recovery

Realised density $\delta = |E|/P$ reads directly as an edge fraction, so $\delta = 0.04$ means 4% of possible pairs are edges. Target density falls short of its request whenever $|A_m| < \lceil dP \rceil$.

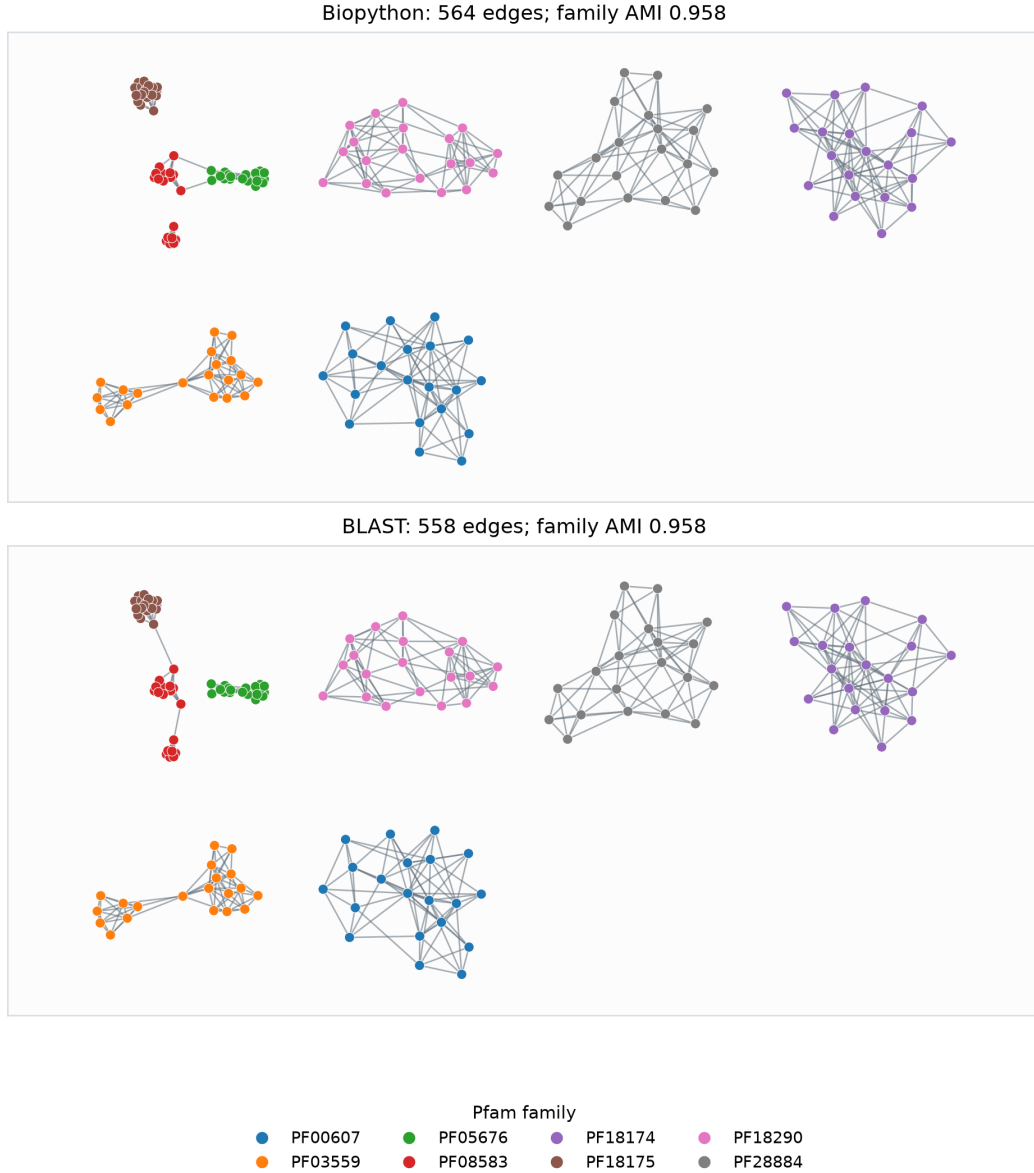


Figure 2: Biopython and BLAST top-5 graphs for collection c4_f2_d20_r001. Colour denotes withheld Pfam family; islands are connected components. Panels share coordinates.

Reference composition fixes vertex count and family mixture. Within this condition, connected-component and Louvain-community counts fell as realised density increased (Fig. 3). At similar density, top- k graphs contained fewer components than percentile and target-density graphs. Top- k allocates local neighbours to every vertex; global rules can concentrate edges among a subset of vertices. Fragmentation split Pfam groups across many inferred communities and reduced AMI.

4.3 AMI as primary agreement measure

NMI and AMI agreed for dense graphs whose inferred community counts approached reference group counts (Fig. 4). Their difference grew under sparse percentile and target-density rules. For BLAST percentile 0.99, median partition contained 150 communities for 8 reference families. Median family NMI was 0.590; median AMI was 0.057. NMI retained agreement produced by many small groups, while AMI subtracted expected agreement conditional on group sizes.

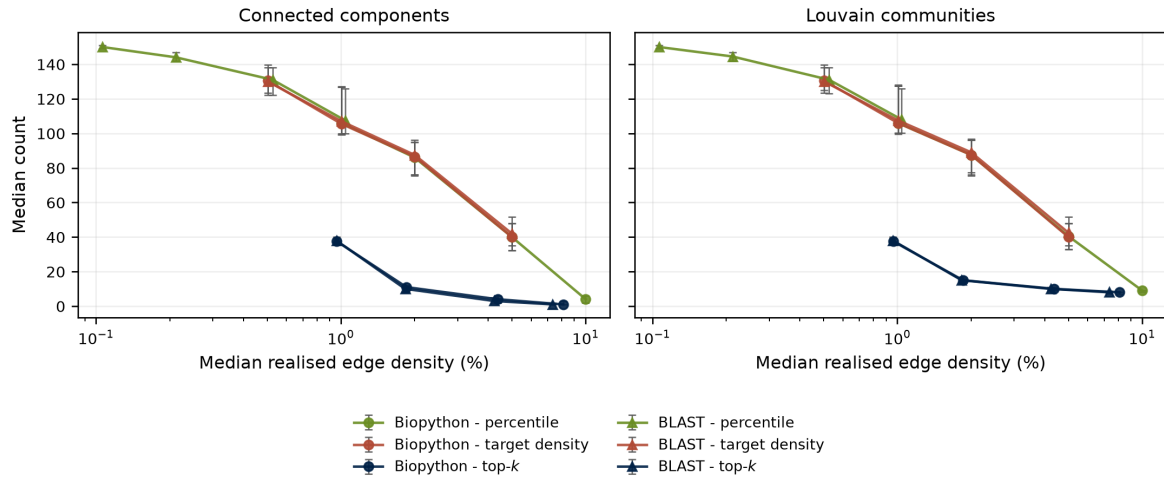


Figure 3: Median component and community counts against realised density for 12 graph settings and 50 reference-condition collections. Colour denotes rule family; marker denotes score method, circle for Biopython and triangle for BLAST. Error bars give interquartile ranges across collections.

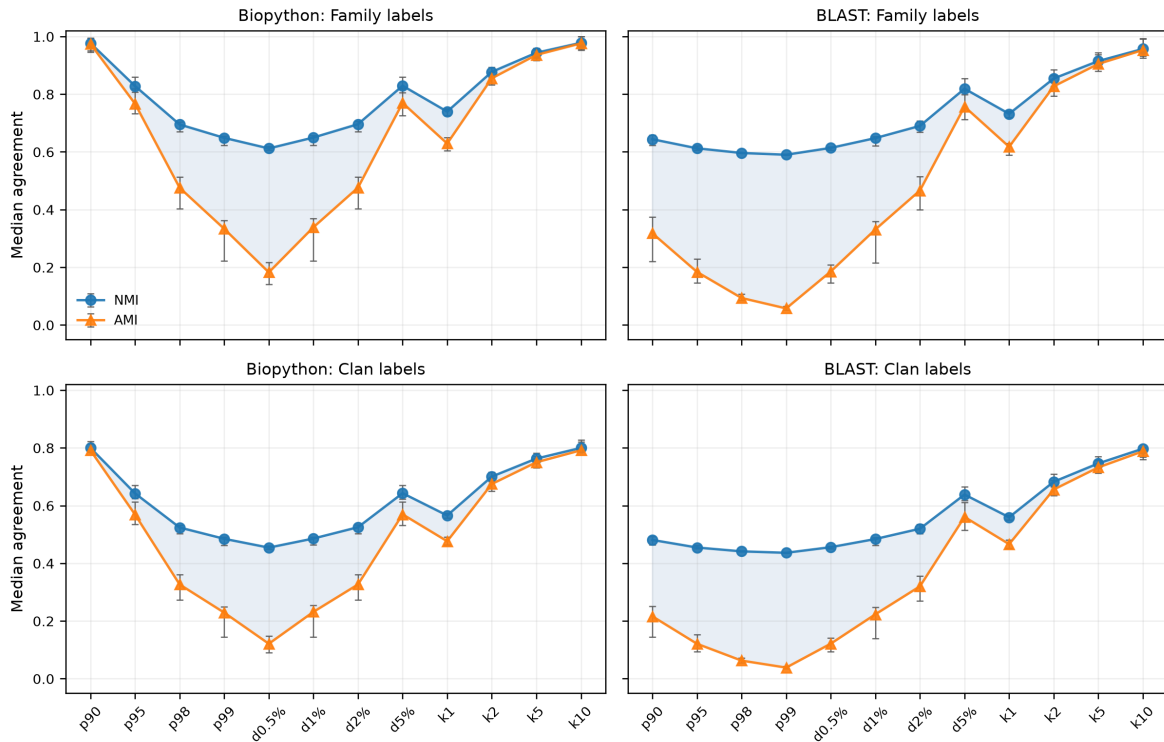


Figure 4: Median NMI and AMI for 12 graph settings in reference condition, with interquartile ranges across collections as error bars. Panels separate score method and withheld label level. Shading marks chance correction. Prefixes p , d , and k denote percentile, target density, and top- k .

Figures 5 and 6 show frequency polygons derived from replicate distributions behind Fig. 4. Each of 50 collections contributes one graph to every score-method and graph-setting combination. Agreement values are placed in 25 equal-width bins on $[0, 1]$. Vertical position gives percentage of collection graphs in each bin; frequencies for each curve sum to 100%. AMI moved sparse-rule distributions towards zero. Primary rule comparisons therefore use AMI; NMI remains a bounded description of unadjusted partition agreement.

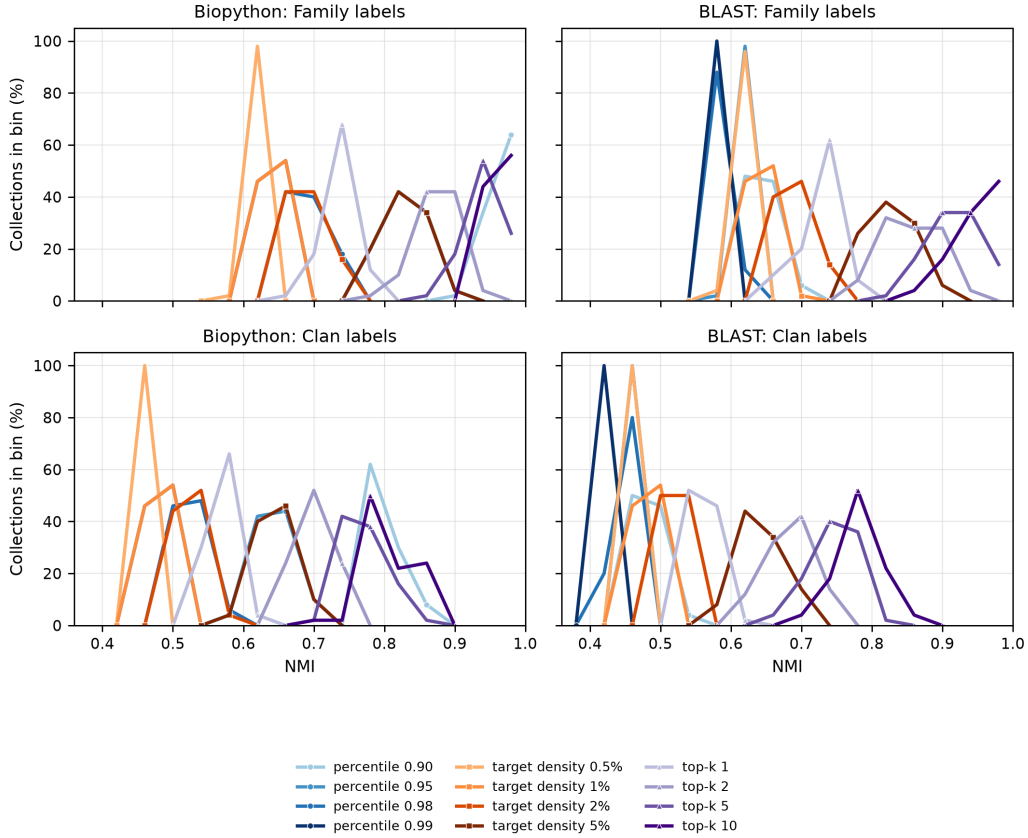


Figure 5: Empirical NMI frequency polygons across 12 graph settings and 50 reference-condition collections. Each curve joins percentages of collection graphs in 25 equal-width NMI bins. Panels separate score method and withheld label level. Line style and marker denote rule family; colour distinguishes parameter values within each family.

4.4 Effect of score method on graph-rule conclusions

Top- k led family recovery under both score methods (Fig. 1). Biopython scored all 460320 master-panel pairs; BLAST reported 19376 pairs, or 4.21%. Among reported pairs, score ranks had Spearman correlation 0.976 (Fig. 7, left). BLAST reports concentrated among pairs with high Biopython scores (Fig. 7, right).

Percentile rules retain a fraction of each method’s available scores. At percentile 0.90, the median realised density was 10.00% for Biopython and 0.83% for BLAST (Fig. 8). Equal percentile parameters therefore represented different edge budgets.

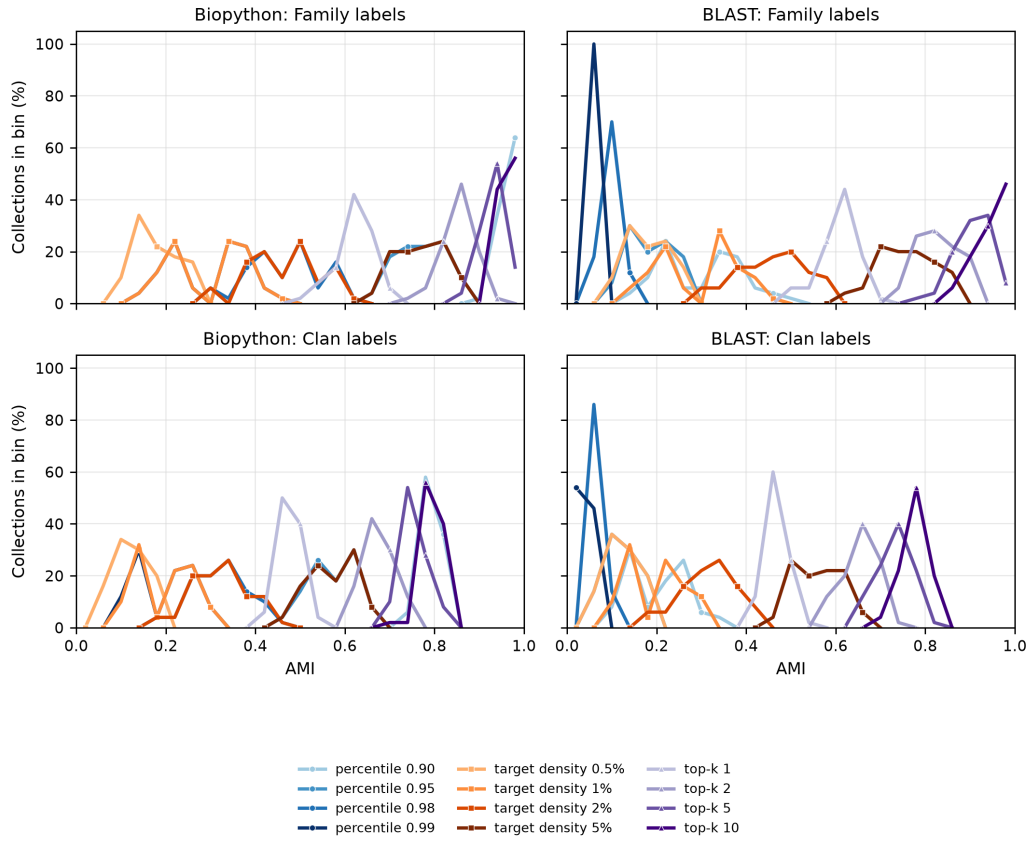


Figure 6: Empirical AMI frequency polygons for graphs in Fig. 5. Panels, curves, bins, and visual encodings follow same design.

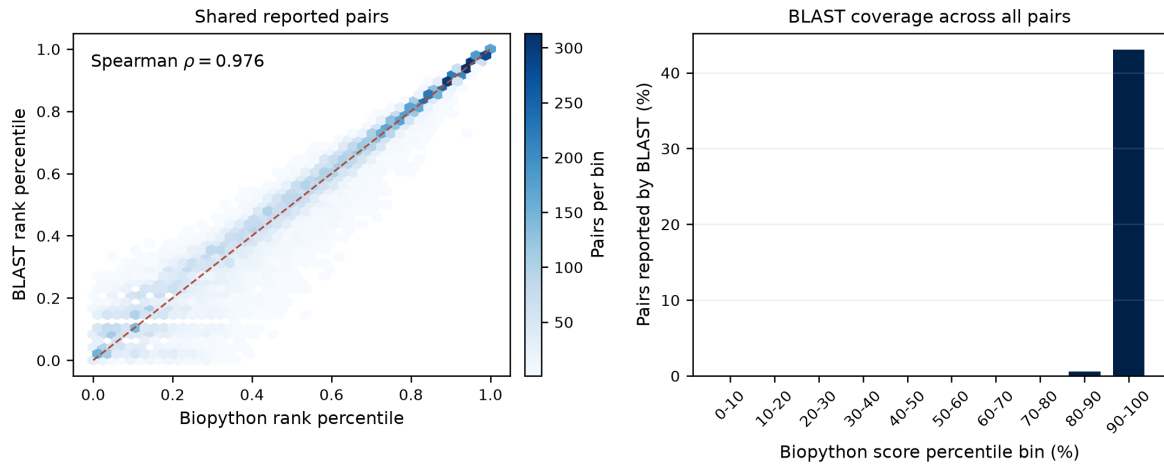


Figure 7: Pair-score comparison on master panel. Left: within-method rank percentiles among 19376 pairs reported by BLAST; colour gives bin count. Right: fraction reported by BLAST within each Biopython score percentile bin across all 460320 pairs.

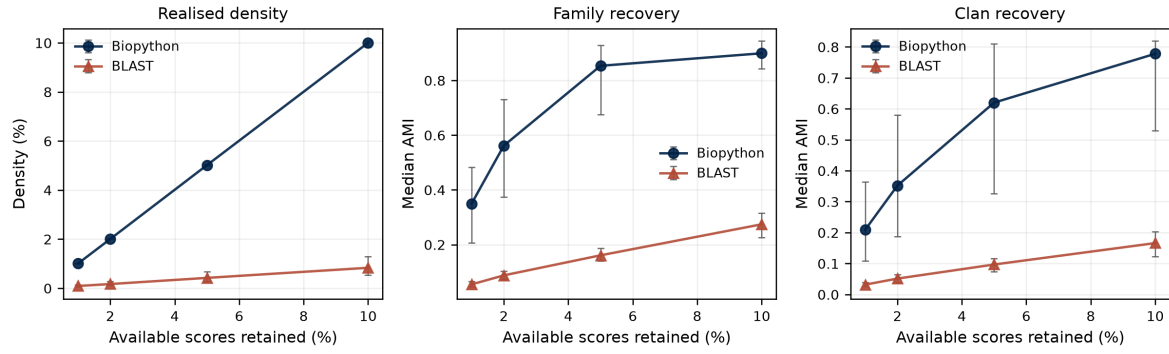


Figure 8: Median density, family AMI, and clan AMI under percentile rules, with interquartile ranges across collections as error bars. Horizontal position is fraction of available scores retained within each method. Summaries use 700 primary collections.

Target density rules compare methods on identical vertex sets and requested edge budgets. Budgets 0.5%, 1%, and 2% were fully supplied, while a 5% request exceeded available BLAST pairs in some collections and was therefore excluded from matched budget interpretation. At 2%, paired median absolute family difference was 0.0034 for NMI and 0.0094 for AMI. Signed differences remained close to zero and their spread increased with density (Fig. 9).

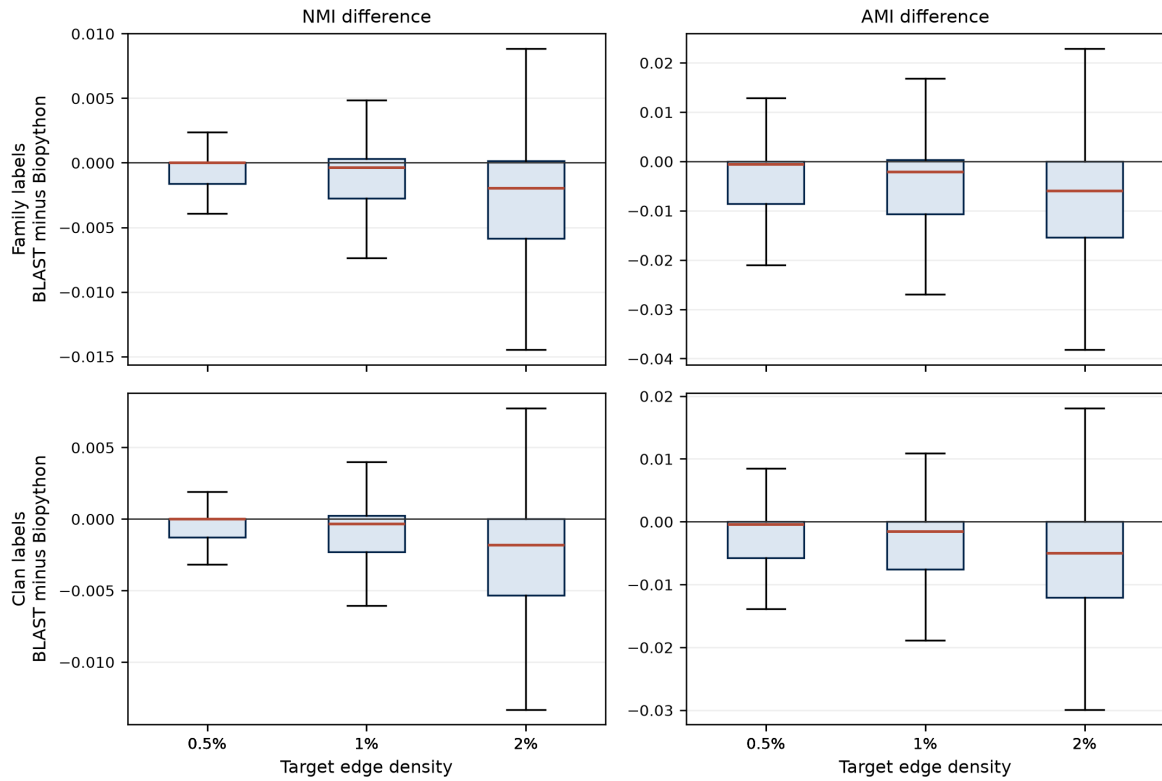


Figure 9: Paired NMI and AMI differences, BLAST minus Biopython, at matched edge densities across 700 collections. Boxes show quartiles and medians; whiskers extend to 1.5 interquartile ranges. Outliers are omitted from display.

4.5 Effect of collection composition under a fixed rule

Top-5 family AMI was evaluated across 14 primary composition conditions, each with 50 collections. Median family AMI ranged from 0.883 to 1.000 for Biopython and from 0.879 to 0.931 for BLAST (Fig. 10). Changes across domain, family, and clan counts were non-monotone. Four saturated design cells remain in audit outputs and do not enter these summaries.

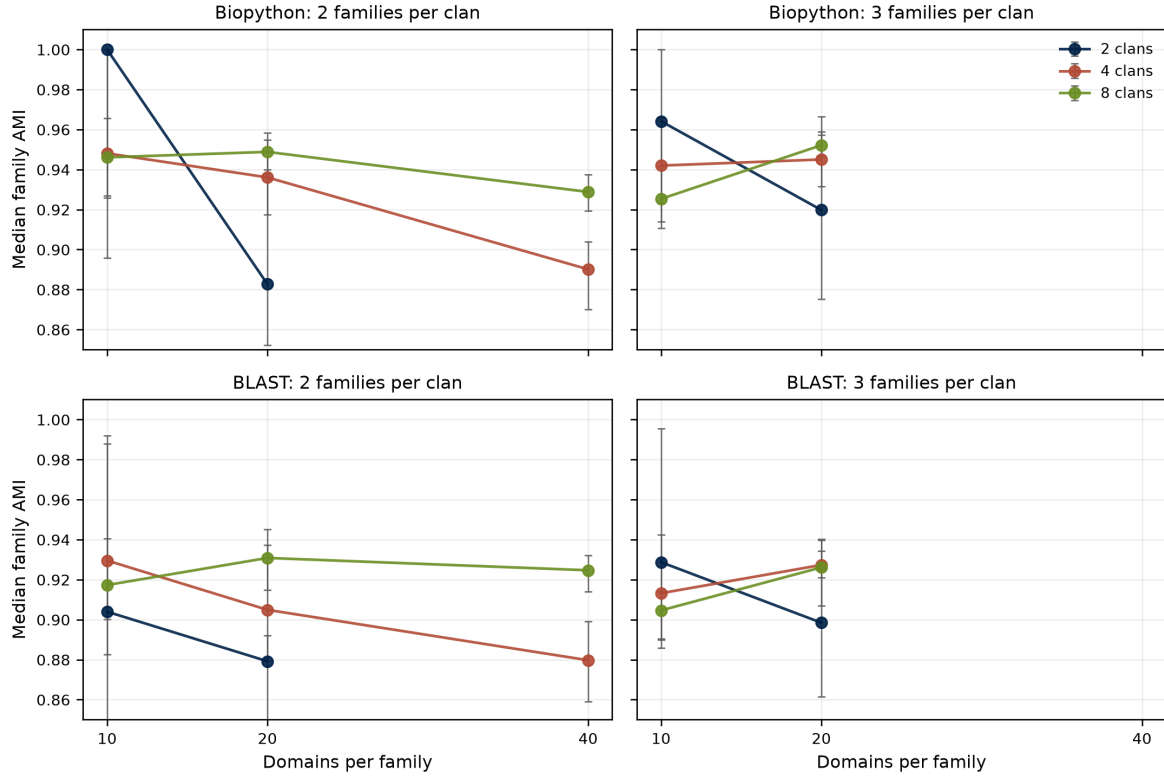


Figure 10: Median family AMI across 14 primary composition conditions under top-5 construction, with interquartile ranges across collections as error bars. Colour gives clan count; panels separate score method and families per clan. Lines connect domain counts.

For fixed k , retained edge count grows at most linearly with vertex count n , while possible pair count grows as $\binom{n}{2}$. Realised density therefore decreased with collection size (Fig. 11, left). Median component count also varied with clan and family counts (Fig. 11, right).

4.6 Sensitivity analyses

We permuted scores within each collection, which preserves the score distribution and rule parameters while randomising pair identities. In reference condition, Biopython top-5 median family AMI changed from 0.936 to 0.009 (Fig. 12, left). Positive null NMI confirmed chance bias in finite partitions addressed by AMI.

For unweighted Louvain, we replaced every retained edge weight by one. Reference Biopython top-5 median family AMI was 0.953, compared with 0.936 under native weights (Fig. 12, right). Selected adjacency therefore retained family agreement after removal of edge weight variation for this setting.

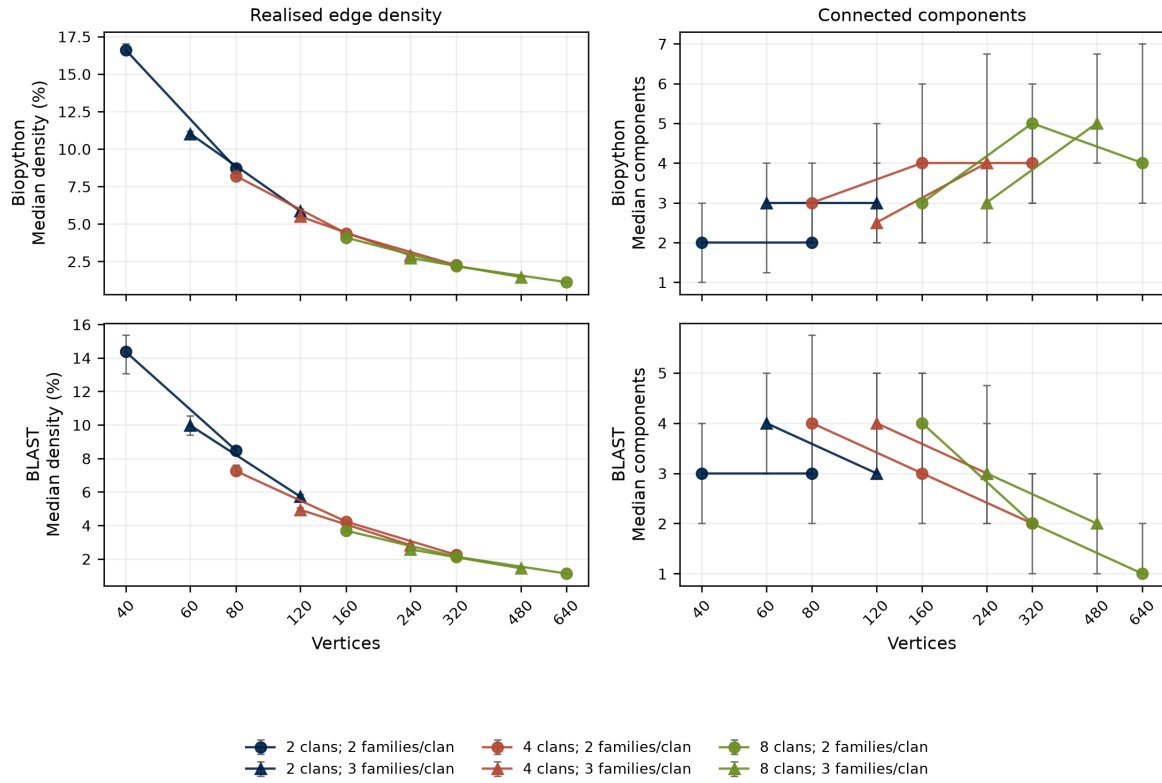


Figure 11: Top-5 graph structure across 14 primary composition conditions, with interquartile ranges across collections as error bars. Rows separate score methods. Colour gives clan count; marker gives families per clan, circle for two and triangle for three. Lines connect domain counts.

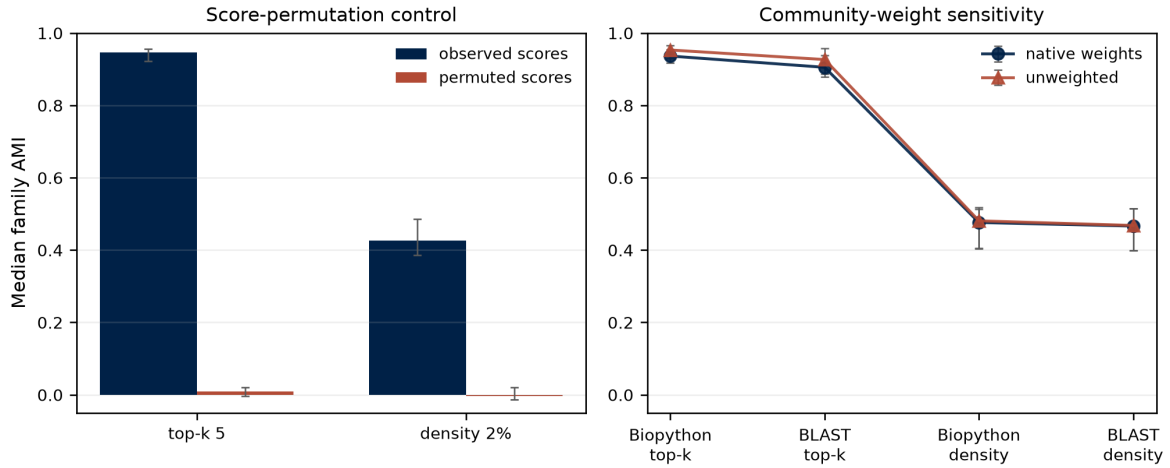


Figure 12: Sensitivity to score permutation and community weight. Bars and markers give medians; error bars give interquartile ranges across collections.

5 Discussion

Main finding. We found that top- k rules, which build a graph by connecting each protein domain to the k domains with which it has the highest alignment scores, recovered withheld Pfam labels most consistently across tested score methods and label levels. Top-10 maximised median family AMI for both methods and was within 0.006 of the highest clan AMI. Top-5 retained fewer edges, with median family AMI 0.938 for Biopython and 0.917 for BLAST. We found that both perform well, so the choice between them depends on the desired graph sparsity.

Edge allocation. From our follow-up experiments on graph topology (Section 4.2), we believe that edge allocation explains much of this result. Sparse global rules produced many components, whereas top- k produced fewer components at similar density because each vertex selects local neighbours. Global density alone therefore did not determine graph structure.

Agreement measure. We found that NMI exceeded AMI under sparse, fragmented settings (Figs. 5 and 6). Because AMI subtracts expected mutual information conditional on observed group sizes, we used AMI for comparisons across rules and compositions, with NMI as an unadjusted bounded descriptor.

Score method. Biopython and BLAST ranks agreed closely among pairs reported by BLAST, while BLAST covered only 4.21% of master pairs. Despite this low coverage, BLAST top- k graphs recovered family labels nearly as well as exact alignment, since BLAST reports concentrated among the highest-scoring pairs (Fig. 7), which are the pairs top- k rules select. Equal percentiles combined score method, coverage, and edge-budget effects. Target density removed the edge-budget difference; at 2% density, median absolute family differences were 0.0034 for NMI and 0.0094 for AMI. Agreement between methods after density matching supports graph-rule conclusions shared by both score systems.

Collection composition. Collection composition changed density, component count, and recovery under a fixed top-5 rule. Balanced sampling estimates behaviour across controlled mixtures rather than natural Pfam abundance. Replicate variation within each condition captures domain and family selection effects for this master panel.

Controls. Our permutation control shows that pair identity carries family information beyond the score distribution: permuting scores reduced reference family AMI from 0.936 to 0.009. With unit weights, family AMI at the reference setting was 0.953, compared with 0.936 under native weights. These controls locate the observed signal primarily in edge placement rather than edge weights for this setting.

Limitations and future work. Inference scope remains balanced mixtures from 24 selected Pfam families. Full-length proteins, alternative databases, synthetic evolutionary data, and structural labels require separate evaluation. 4 saturated design conditions reduced primary factorial design to 14 conditions. Louvain resolution and stochasticity received limited study. DEDAL lacks scaled evidence

here. Future work should repeat sampling across broader Pfam panels, model composition effects with hierarchical uncertainty, test graph filtrations across continuous edge budgets, and compare community algorithms. These experiments estimate stability across panels and graph construction choices.

6 Conclusion

In this project, we studied sequence-similarity networks, graphs built from protein sequences using pairwise alignment scores. Our goal was to develop and test a framework that evaluates how methodological choices in building SSNs affect the biological relevance of the resulting networks. We built a pipeline that maps recorded domain collections to pair scores, weighted graphs, communities, and auditable agreement measures. Among tested settings, we found that top- k rules recovered withheld Pfam structure most consistently; top-10 maximised family AMI and top-5 supplied a sparser alternative with median family AMI of at least 0.917 under both score methods. AMI corrected chance agreement caused by variable community counts. Biopython and BLAST produced similar recovery after edge budgets were matched, while equal percentiles produced different graph densities. Sampling frame, score coverage, edge rule, and realised density should accompany each reported SSN result.

A Panel selection algorithm

Parameters used were $c^* = 8$, $f^* = 3$, $r^* = 40$, and $\sigma = 20260828$.

Algorithm 1 Seeded selection of clan and family panel

Require: catalogue of Pfam families with clan assignment; targets c^* clans and f^* families per clan; minimum reported seed count r^* ; seed σ

Ensure: panel of c^*f^* families, and trace of every family inspected

```
1:  $C \leftarrow$  clans holding at least  $f^*$  catalogue families, excluding unassigned
2: sort  $C$ , then shuffle under  $\sigma$ 
3:  $P \leftarrow \emptyset$ 
4: for all  $C \in C$  in shuffled order do
5:   sort families of  $C$ , then shuffle under same stream
6:    $A \leftarrow \emptyset$ 
7:   for all  $F$  in shuffled families of  $C$  do
8:     query InterPro for entry type and reported seed count of  $F$ 
9:     append  $F$  to trace
10:    if type of  $F$  is domain or family, and seed count  $\geq r^*$  then
11:       $A \leftarrow A \cup \{F\}$ 
12:    end if
13:    if  $|A| = f^*$  then
14:      break
15:    end if
16:  end for
17:  if  $|A| = f^*$  then
18:     $P \leftarrow P \cup A$  ▷ otherwise clan  $C$  is skipped
19:    if  $P$  spans  $c^*$  clans then
20:      break
21:    end if
22:  end if
23: end for
24: return  $P$  and trace
```

References

- S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, 1990. doi: 10.1016/S0022-2836(05)80360-2. URL [https://doi.org/10.1016/S0022-2836\(05\)80360-2](https://doi.org/10.1016/S0022-2836(05)80360-2).
- L. Apeltsin, J. H. Morris, P. C. Babbitt, and T. E. Ferrin. Improving the quality of protein similarity network clustering algorithms using the network edge weight distribution. *Bioinformatics*, 27(3):326–333, 2011. doi: 10.1093/bioinformatics/btq655. URL <https://doi.org/10.1093/bioinformatics/btq655>.
- H. J. Atkinson, J. H. Morris, T. E. Ferrin, and P. C. Babbitt. Using sequence similarity networks for visualization of relationships across diverse protein superfamilies. *PLOS ONE*, 4(2):e4345, 2009. doi: 10.1371/journal.pone.0004345. URL <https://doi.org/10.1371/journal.pone.0004345>.
- V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, 2008. doi: 10.1088/1742-5468/2008/10/P10008. URL <https://doi.org/10.1088/1742-5468/2008/10/P10008>.
- C. Camacho, G. Coulouris, V. Avagyan, N. Ma, J. Papadopoulos, K. Bealer, and T. L. Madden. BLAST+: Architecture and applications. *BMC Bioinformatics*, 10:421, 2009. doi: 10.1186/1471-2105-10-421. URL <https://doi.org/10.1186/1471-2105-10-421>.

- O. Camoglu, T. Can, and A. K. Singh. Integrating multi-attribute similarity networks for robust representation of the protein space. *Bioinformatics*, 22(13):1585–1592, 2006. doi: 10.1093/bioinformatics/btl130. URL <https://doi.org/10.1093/bioinformatics/btl130>.
- P. J. A. Cock, T. Antao, J. T. Chang, B. A. Chapman, C. J. Cox, A. Dalke, I. Friedberg, T. Hamelryck, F. Kauff, B. Wilczynski, and M. J. L. de Hoon. Biopython: Freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009. doi: 10.1093/bioinformatics/btp163. URL <https://doi.org/10.1093/bioinformatics/btp163>.
- R. Durbin, S. R. Eddy, A. Krogh, and G. Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, Cambridge, 1998.
- A. J. Enright, S. Van Dongen, and C. A. Ouzounis. An efficient algorithm for large-scale detection of protein families. *Nucleic Acids Research*, 30(7):1575–1584, 2002. doi: 10.1093/nar/30.7.1575. URL <https://doi.org/10.1093/nar/30.7.1575>.
- J. A. Gerlt, J. T. Bouvier, D. B. Davidson, H. J. Imker, B. Sadkhin, D. R. Slater, and K. L. Whalen. Enzyme function initiative-enzyme similarity tool (EFI-EST): A web tool for generating protein sequence similarity networks. *Biochimica et Biophysica Acta (BBA) - Proteins and Proteomics*, 1854(8):1019–1037, 2015. doi: 10.1016/j.bbapap.2015.04.015. URL <https://doi.org/10.1016/j.bbapap.2015.04.015>.
- O. Gotoh. An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3):705–708, 1982. doi: 10.1016/0022-2836(82)90398-9. URL [https://doi.org/10.1016/0022-2836\(82\)90398-9](https://doi.org/10.1016/0022-2836(82)90398-9).
- A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, and J. Millman, editors, *Proceedings of the 7th Python in Science Conference*, pages 11–15, Pasadena, California, 2008. URL <https://proceedings.scipy.org/articles/PFVC8793.pdf>.
- S. Henikoff and J. G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992. doi: 10.1073/pnas.89.22.10915. URL <https://doi.org/10.1073/pnas.89.22.10915>.
- B. V. H. Hornung and N. Terrapon. An objective criterion to evaluate sequence-similarity networks helps in dividing the protein family sequence space. *PLOS Computational Biology*, 19(8):e1010881, 2023. doi: 10.1371/journal.pcbi.1010881. URL <https://doi.org/10.1371/journal.pcbi.1010881>.
- S. Karlin and S. F. Altschul. Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes. *Proceedings of the National Academy of Sciences*, 87(6):2264–2268, 1990. doi: 10.1073/pnas.87.6.2264. URL <https://doi.org/10.1073/pnas.87.6.2264>.
- F. Llinares-López, Q. Berthet, M. Blondel, O. Teboul, and J.-P. Vert. Deep embedding and alignment of protein sequences. *Nature Methods*, 20:104–111, 2023. doi: 10.1038/s41592-022-01700-2. URL <https://doi.org/10.1038/s41592-022-01700-2>.
- J. Mistry, S. Chuguransky, L. Williams, M. Qureshi, G. A. Salazar, E. L. L. Sonnhammer, S. C. E. Tosatto, L. Paladin, S. Raj, L. J. Richardson, R. D. Finn, and A. Bateman. Pfam: The protein families database in 2021. *Nucleic Acids Research*, 49(D1):D412–D419, 2021. doi: 10.1093/nar/gkaa913. URL <https://doi.org/10.1093/nar/gkaa913>.
- F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL <https://www.jmlr.org/papers/v12/pedregosa11a.html>.
- T. F. Smith and M. S. Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1):195–197, 1981. doi: 10.1016/0022-2836(81)90087-5. URL [https://doi.org/10.1016/0022-2836\(81\)90087-5](https://doi.org/10.1016/0022-2836(81)90087-5).
- N. X. Vinh, J. Epps, and J. Bailey. Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. *Journal of Machine Learning Research*, 11:2837–2854, 2010. URL <https://www.jmlr.org/papers/v11/vinh10a.html>.